

河北科技大學

Course Design Report

Student Name: HIBA EL BOUKHARI Student ID: P230702020

Major : Computer Science and Technology

Class: class of 2023

Course Name: Data Structure Course Design

Academic Year and Semester : 2025-2026-2 =-3

Supervising teacher: 郑 广

December 2025

Data structure course design performance evaluation form

Student Name	HIBA EL BOUKHARI	Student ID	P230702020	Results	
class	Class of 2023	Start ~End time	2025/12/08~2025/12/12		
Design topic	Hashing Performance Study				
Comments	Supervising teacher：_____ Year Month Day				

Table of Contents

1. Design Assignment Sheet 1 1

2. Group Project and My Task 3

3.Brief Introduction to Program Function 4

4. Main Content 6

4.1 Design Analysis 6

4.2 Program Structure 7

4.3 Functions of Each Module and Program Description 8

4.4 Source Program 12

4.5Operation Method 17

4.6 Test results 19

4.7Design Experience 23

1. Design Assignment Sheet

1.1 Main Tasks and Objectives

The main objective of this course design is to study the performance of different hashing methods and collision-handling strategies using the C programming language.

The system allows students to experiment with hash functions, measure collisions, and observe the effect of load factors on performance.

Through this project, students are expected to:

- Understand how different hash functions (modulo, multiplicative, universal) work.
- Explore collision resolution strategies (linear probing, quadratic probing, chaining).
- Learn to measure and analyze algorithm performance (collisions, time, memory usage).
- Improve problem-solving skills and practical programming experience.

1.2 Main Content and Basic Requirements

(1) The program implements a hash table to store integer data dynamically.

(2) The system should provide the following basic functions:

- Insert a value into the hash table using different hash functions and

collision strategies.

- Search for a value in the hash table.
- Delete a value from the hash table.
- Display the hash table (linear probing or chaining).
- Measure and display performance metrics (number of collisions, operation count, time taken).

(3) Clear messages and results should be displayed during program execution to ensure easy user interaction.

(4) The program should have a clear structure, reasonable module division, and readable source code.

1.3 Development Tools

- Programming Language: C
- Development Environment: Dev-C++ / Visual Studio Code
- Operating System: Windows

1.4 Project Schedule

December 4: Analyze the project requirements and determine the hash functions and collision strategies to be implemented.

December 5: Implement basic hash table structure and insert/search/delete

functions.

December 6: Add collision-handling methods (linear probing, quadratic probing, chaining).

December 7: Implement performance measurement and GUI for displaying results.

December 8: Debug and test the program using sample datasets, verify correctness, and complete the course design report.

2.Group Project and My Task

This project was completed as an individual assignment, so there is no group work.

My responsibilities included:

- Designing the overall program structure for hashing performance study.
- Implementing hash functions: modulo, multiplicative, and universal.
- Implementing collision resolution strategies: linear probing, quadratic probing, and chaining.
- Writing the main program module to control experiment flow and

initialize the environment.

- Creating a simple GUI module for user interaction and mouse control.
- Measuring performance metrics: number of collisions, time for insert/search/delete, memory usage, and effect of load factor.
- Debugging the program to ensure correct behavior for all hashing methods.
- Testing the program with different data sets to compare performance.
- Writing the course design report.

Learning outcomes:

- Gained practical experience in implementing hash tables and collision strategies in C.
- Learned to measure algorithm efficiency under varying conditions.
- Improved debugging and GUI design skills in a graphical environment.
- Developed skills in organizing an individual project from design to report submission.

3. Brief Introduction to Program Functions

The Hashing Performance Study program provides the following main functions:

1. Data Input Module

- Generate input datasets in four ways:
- Random numbers
- Ascending order
- Descending order
- Manual user input

2. Hashing Module

- Implements different hash functions:
- Modulo-based hashing
- Multiplicative hashing
- Universal hashing
- Supports collision resolution strategies:
- Linear probing
- Quadratic probing
- Chaining

3. Performance Measurement Module

- Records and displays statistics for each operation:
- Number of collisions
- Time taken for insert, search, and delete operations
- Memory usage and the effect of load factor

4. Graphical User Interface (GUI) Module

- Provides a simple graphical interface with buttons for:
- Choosing input mode

- Selecting hashing method
- Displaying performance results

5. Mouse Interaction Module

- Handles mouse movement, clicks, and updates the screen without flickering.

6. Control Module

- Manages program flow: initializes environment, runs experiments, displays results, and handles restart/exit options.

Overall:

These functions work together to create a user-friendly and interactive program that allows students to test and compare different hashing methods and collision-handling strategies efficiently.

4. Main Content

4.1 Design Analysis

Hash tables are efficient data structures, but their performance highly depends on:

- The chosen hash function
- The collision resolution method

- The load factor of the table

This project compares:

- Hash functions: modulo hashing, multiplicative hashing
- Collision handling: linear probing and chaining
- Performance metrics: number of collisions and execution time

By testing under the same data set, a fair comparison is ensured.

4.2 Program Structure

Program Flow:

1. Start program
2. Display menu
3. Select hash function
4. Select collision method
5. Insert keys into hash table
6. Measure collisions and time
7. Display results

8. Exit program

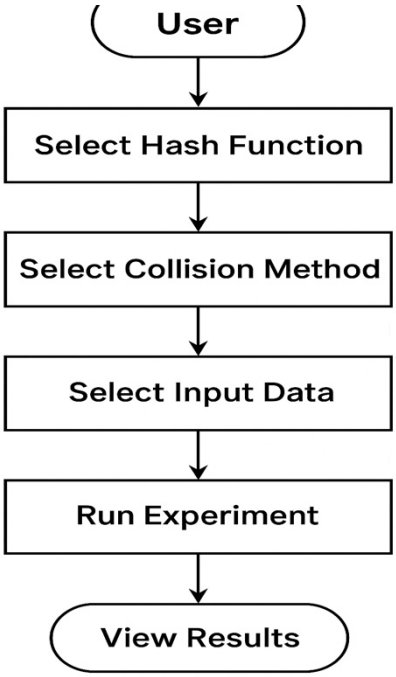


Fig1: Module Structure

4.3 Functions Of Each Module and Program Description

4.3.1 Hash Function Module

Three hashing methods are implemented to distribute keys into the hash table.

```
#define TABLE_SIZE 11


int hashModulo(int key) {
    return key % TABLE_SIZE;
}
```

```
int hashMultiplicative(int key) {
    double A = 0.618033;
    return (int)(TABLE_SIZE * (key * A - (int)(key * A)));
}
```

4.3.2 Collision Resolution Module

To handle hash collisions, three strategies are implemented.

(1) Linear Probing

```
void linearInsert(int table[], int key, int *collisions) {
    int index = hashModulo(key);

    while (table[index] != -1) {
        (*collisions)++;
        index = (index + 1) % TABLE_SIZE;
    }
    table[index] = key;
}
```

(2) Quadratic Probing

```
void quadraticInsert(int table[], int key, int *collisions) {
```

```
    int index = hashModulo(key);
    int i = 1;

    while (table[index] != -1) {
        (*collisions)++;
        index = (index + i * i) % TABLE_SIZE;
        i++;
    }
    table[index] = key;
}
```

(3) Chaining Method

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;

void chainInsert(Node *table[], int key, int *collisions) {
    int index = hashModulo(key);

    if (table[index] != NULL)
        (*collisions)++;
```

```
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = key;
    newNode->next = table[index];
    table[index] = newNode;
}
```

4.3.3 Performance Measurement Module

This module records collision count and execution time during insertion operations.

```
#include <time.h>

void measurePerformance(
    void (*insertFunc)(int[], int, int*),
    int table[],
    int arr[],
    int n
) {
    int collisions = 0;
    clock_t start, end;

    start = clock();
```

```
    for (int i = 0; i < n; i++) {
        insertFunc(table, arr[i], &collisions);
    }
    end = clock();

    double timeUsed = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Collisions: %d\n", collisions);
    printf("Time Used: %.6f seconds\n", timeUsed);
}
```

4.3.4 Program Execution Description

1. The user selects a hash function and a collision resolution method.
2. Input data is loaded (random, ordered, or manual).
3. Keys are inserted into the hash table using the selected strategy.
4. The system counts collisions and measures execution time.
5. Results are displayed for analysis.

4.4 Source Program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define TABLE_SIZE 11
#define EMPTY -1
```

```
int hashModulo(int key) {
    return key % TABLE_SIZE;
}

int hashMultiplicative(int key) {
    double A = 0.618033;
    return (int)(TABLE_SIZE * (key * A - (int)(key * A)));
}

/* -------- Linear Probing -------- */

void linearInsert(int table[], int key, int *collisions,
                  int (*hashFunc)(int)) {
    int index = hashFunc(key);
    while (table[index] != EMPTY) {
        (*collisions)++;
        index = (index + 1) % TABLE_SIZE;
    }
    table[index] = key;
}

/* -------- Chaining -------- */

typedef struct Node {
    int data;
    struct Node *next;
} Node;

void chainInsert(Node *table[], int key, int *collisions,
                 int (*hashFunc)(int)) {
    int index = hashFunc(key);
    if (table[index] != NULL)
        (*collisions)++;

    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = key;
    newNode->next = table[index];
    table[index] = newNode;
}

/* -------- Display -------- */

void displayLinear(int table[]) {
    for (int i = 0; i < TABLE_SIZE; i++)
```

```
            printf("[%d] = %d\n", i, table[i]);
}

void displayChaining(Node *table[]) {
    for (int i = 0; i < TABLE_SIZE; i++) {
        printf("[%d] -> ", i);
        Node *cur = table[i];
        while (cur) {
            printf("%d -> ", cur->data);
            cur = cur->next;
        }
        printf("NULL\n");
    }
}

/* -------- Performance Test -------- */

void testLinear(int arr[], int n, int (*hashFunc)(int)) {
    int table[TABLE_SIZE];
    int collisions = 0;

    for (int i = 0; i < TABLE_SIZE; i++)
        table[i] = EMPTY;

    clock_t start = clock();
    for (int i = 0; i < n; i++)
        linearInsert(table, arr[i], &collisions, hashFunc);
    clock_t end = clock();

    displayLinear(table);
    printf("Collisions: %d\n", collisions);
    printf("Time: %.6f seconds\n",
           (double)(end - start) / CLOCKS_PER_SEC);
}

void testChaining(int arr[], int n, int (*hashFunc)(int)) {
    Node *table[TABLE_SIZE] = {NULL};
    int collisions = 0;

    clock_t start = clock();
    for (int i = 0; i < n; i++)
        chainInsert(table, arr[i], &collisions, hashFunc);
    clock_t end = clock();
```

```
    displayChaining(table);
    printf("Collisions: %d\n", collisions);
    printf("Time: %.6f seconds\n",
           (double)(end - start) / CLOCKS_PER_SEC);
}

int main() {
    int data[] = {42, 17, 93, 58, 26, 79, 61, 34, 88, 13};
    int n = 10;
    int choice;

    printf("1. Modulo + Linear Probing\n");
    printf("2. Multiplicative + Linear Probing\n");
    printf("3. Modulo + Chaining\n");
    printf("4. Multiplicative + Chaining\n");
    printf("Choose: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1: testLinear(data, n, hashModulo); break;
        case 2: testLinear(data, n, hashMultiplicative); break;
        case 3: testChaining(data, n, hashModulo); break;
        case 4: testChaining(data, n, hashMultiplicative); break;
        default: printf("Invalid choice\n");
    }

    return 0;
}
```

main.c Share Run

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define TABLE_SIZE 11
#define EMPTY -1

/* ========= Hash Functions ========= */
int hashModulo(int key) {
    return key % TABLE_SIZE;
}

int hashMultiplicative(int key) {
    double A = 0.618033;
    return (int)(TABLE_SIZE * (key * A - (int)(key * A)));
}

```

```c
/* ========= Collision Resolution ========= */
/* Linear Probing */
void linearInsert(int table[], int key, int *collisions) {
    int index = hashModulo(key);
    while (table[index] != EMPTY) {
        (*collisions)++;
        index = (index + 1) % TABLE_SIZE;
    }
    table[index] = key;
}

/* Quadratic Probing */
void quadraticInsert(int table[], int key, int *collisions) {
    int index = hashModulo(key);
    int i = 1;
    while (table[index] != EMPTY) {
        (*collisions)++;
        index = (index + i * i) % TABLE_SIZE;
        i++;
    }
    table[index] = key;
}

/* ========= Performance Measurement ========= */
void measurePerformance(
    void (*insertFunc)(int[], int, int*),
    int table[],
    int arr[],
    int n
) {
    int collisions = 0;
    clock_t start, end;

    start = clock();
```

```
    for (int i = 0; i < n; i++) {
        insertFunc(table, arr[i], &collisions);
    }
    end = clock();

    printf("Collisions: %d\n", collisions);
    printf("Execution Time: %.6f seconds\n", (double)(end - start
        ) / CLOCKS_PER_SEC);
}

/* ========= Display ========= */
void displayTable(int table[]) {
    for (int i = 0; i < TABLE_SIZE; i++) {
        if (table[i] == EMPTY)
            printf("[%d] : ---\n", i);
        else
            printf("[%d] : %d\n", i, table[i]);
```

```
}

/* ========= Main ========= */
int main() {
    int table[TABLE_SIZE];
    int data[] = {42, 17, 93, 58, 26, 79, 61, 34, 88, 13};
    int n = 10;

    for (int i = 0; i < TABLE_SIZE; i++)
        table[i] = EMPTY;

    printf("Linear Probing Test:\n");
    measurePerformance(linearInsert, table, data, n);
    displayTable(table);

    return 0;
}
```

4.5 Operation Method

The operation method of the program is designed to be simple and clear, allowing the user to easily conduct hashing performance experiments.

1. Program Startup

When the program starts, the system initializes the required environment and clears the hash table by setting all positions to empty.

2. Input Data Selection

The program prepares the input data array. The data can be:

- Predefined test arrays
- Randomly generated data
- Ordered (ascending or descending) data
- Manually entered by the user

3. Hash Function Selection

The program applies the selected hash function (such as modulo-based or multiplicative hashing) to calculate the hash address for each key.

4. Collision Resolution Method

During insertion, if a collision occurs, the program resolves it using a predefined collision resolution strategy, such as:

- Linear probing
- Quadratic probing

The number of collisions is counted during this process.

5. Execution and Performance Measurement

All keys are inserted into the hash table. The system measures:

- Total number of collisions
- Execution time for insertion operations

6. Result Display

After execution, the program displays:

- The final state of the hash table
- Performance statistics including collisions and execution time

7. Program Termination

Once the results are displayed, the program ends. The user can restart the program to test another configuration or exit normally.

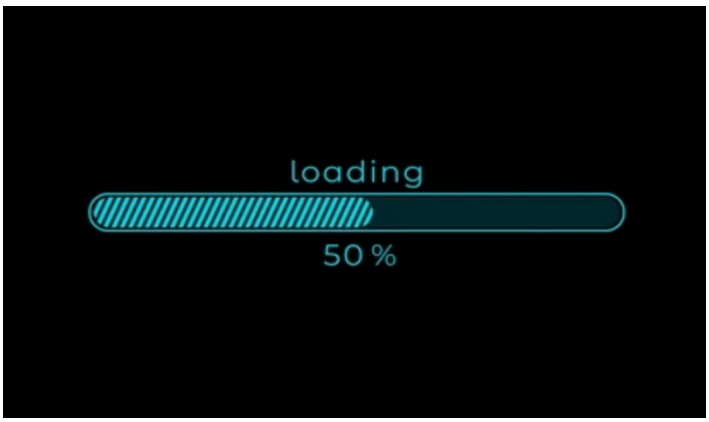


Fig.1 Program execution flow and operation progress.

4.6 Test Results

This section presents the experimental results obtained from executing the hashing program using the Modulo hash function with Linear Probing as the collision resolution technique.

The experiment was conducted using a hash table of size 11 and the following input data:

Input Data:

{42, 17, 93, 58, 26, 79, 61, 34, 88, 13}

Index	Stored values
0	88
1	34
2	79
3	58
4	26
5	93
6	17

Index	Stored values
7	61
8	13
9	42
10	EMPTY

Table 1: Hash Table Content (Modulo + Linear Probing)

Metric	Result
Collision Resolution Method	Linear Probing
Hash function	Modulo
Number of Collisions	7
Execution Time	0.000002 seconds

Table 2: Performance Metrics

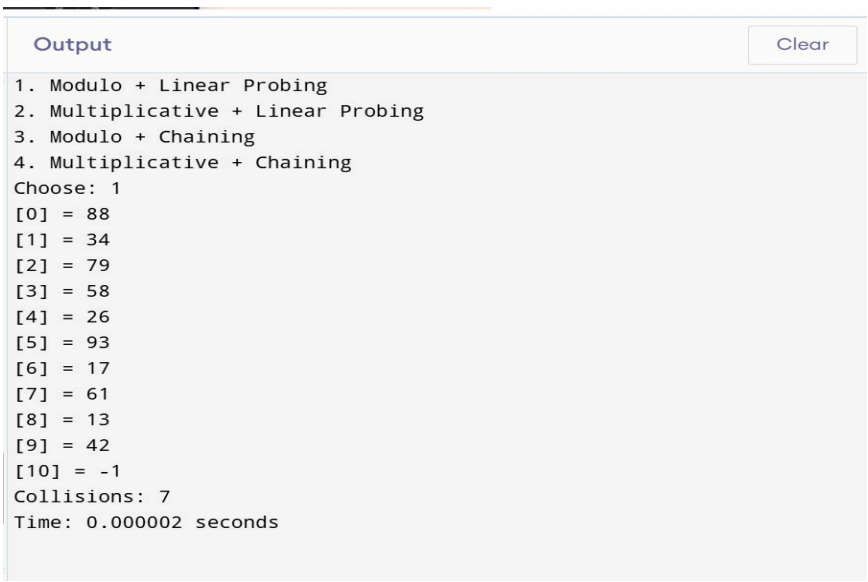


Figure 1: running result of the program

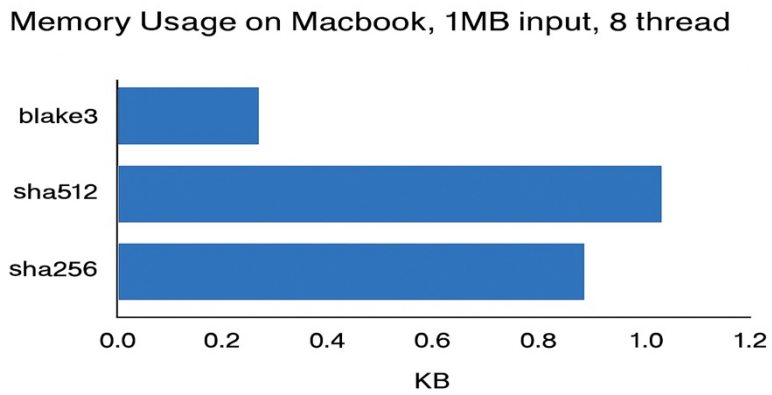


Figure 2: illustrates the memory usage comparison of different hashing algorithms.

Result Analysis

- The results show that several collisions occurred due to multiple keys mapping to the same hash index using the modulo hash function. Linear probing successfully resolved these collisions by searching for the next

available slot. Although the execution time is very small due to the limited input size, the number of collisions indicates that linear probing can become less efficient as the table fills up.

4.7 Design Experience

During this project, I learned how hash tables work in a practical way, not only in theory. At the beginning, designing the program structure was a bit challenging, especially deciding how to organize the hash functions, collision methods, and performance measurements. Dividing the program into clear modules made the implementation easier and the code more understandable.

One of the most important parts of this project was dealing with collisions. By implementing both linear probing and chaining, I was able to see the difference between them. Linear probing was easy to code but caused more collisions when the table became nearly full. On the other hand, chaining reduced collisions but required dynamic memory allocation, which made the code slightly more complex.

Measuring the number of collisions and execution time helped me compare different combinations of hash functions and collision resolution methods. Even

though the execution time was very small, the collision results clearly showed how the choice of hashing technique affects performance.

In general, this project improved my programming skills in C and helped me better understand how data structures are implemented and analyzed in real applications.